MQAT

Exercises in R

Rosa Félix Gonçalo F. Matos Gabriel Valença
Filipe Moura

2026-10-12

Table of contents

1. Quantitative Methods of Analysis in Transportation course **1**

2. Introduction **3**
Why R and GIS . . . 3
Recommended readings . . . 3

3. Software **5**
3.1. R . . . 5
3.1.1. Windows . . . 5
3.1.2. Mac . . . 5
3.1.3. Ubuntu . . . 6
3.2. IDE . . . 7
3.2.1. Positron . . . 7
3.2.2. Windows . . . 7
3.2.3. MacOS . . . 8
3.2.4. Ubuntu . . . 8
3.2.5. RStudio . . . 8
3.2.6. Windows . . . 8
3.2.7. MacOS . . . 8
3.2.8. Ubuntu . . . 8
3.3. R packages . . . 8
3.4. Cheat sheets . . . 10

I. Data and Models **13**

4. R basics **15**
4.1. Math operations . . . 15
4.1.1. Sum . . . 15
4.1.2. Subtraction . . . 15
4.1.3. Multiplication . . . 15
4.1.4. Division . . . 15
4.1.5. Round the number . . . 16
4.2. Basics . . . 16
4.2.1. Combinations . . . 16
4.2.2. Create a comment with `ctrl + shift + c` . . . 16
4.2.3. Create a table . . . 17
4.3. Practical exercise . . . 17
4.3.1. Import dataset . . . 18
4.3.2. Take a first look at the data . . . 18

4.3.3. Explore the data . . . 19
4.3.4. Modify original data . . . 20
4.3.5. Export data . . . 21
4.3.6. Import data . . . 21

5. Data manipulation **23**
5.1. Select variables . . . 24
5.1.1. Using pipes! . . . 24
5.2. Filter observations . . . 25
5.3. Create new variables . . . 25
5.4. Change data type . . . 25
5.4.1. Factors . . . 26
5.5. Join data tables . . . 28
5.6. group_by and summarize . . . 30
5.7. Arrange data . . . 31
5.8. All together now! . . . 32
5.9. Other dplyr functions . . . 33

6. Exploratory Data Analysis **35**
6.1. Intro . . . 35
6.2. Load packages . . . 35
6.3. Dataset . . . 35
6.3.1. Import dataset . . . 36
6.4. Summary statistics . . . 37
6.5. Missing data . . . 39
6.6. Outliers . . . 39
6.6.1. Treating outliers . . . 40
6.7. Histograms . . . 42
6.8. Correlations . . . 48

References **55**

1. Quantitative Methods of Analysis in Transportation course

Materials, exercises, data and tutorials for the **Quantitative Methods of Analysis in Transportation** course of the MSc in Transportation Systems at Instituto Superior Técnico - University of Lisbon.



TÉCNICO
UNIVERSIDADE
DE LISBOA

MESTRADO
EM SISTEMAS DE
TRANSPORTES

This is not the official book for the course.

Please refer to Fénix: Quantitative Methods of Analysis in Transportation

2. Introduction

This book aims to provide tools to deal with transportation datasets using R programming, an open-source and widely used tool for data analytics in urban mobility.

It includes course sporting materials, exercises, and examples to explore with **R**, with a focus on modelling and spatial data analysis using GIS techniques.

Why R and GIS

Most academic programs focus on teaching modelling and deep analysis of data. However, there is a need to learn how to explore and prepare a dataset for modelling. The use of **programming and GIS** techniques have enormous advantages, including their flexibility; reproducibility; and transparency and understanding the step-by-step process.

The use of GIS techniques in transportation is of enormous relevance when doing accessibility analysis or reeling with georreferenced transportation data, such as bike sharing route trips' datasets, origin-destination flows datasets, home/work locations, GTFS public transit data, and so on. There is a need to learn how to locate these open datasets, how to explore them and how to integrate them into transportation and urban analysis. Additionally, the use of open source software and datasets allows researchers to perform methods that are reproducible and transparent.

TLDR

- Open-source tools widely used in data analytics and spatial analysis
- Flexibility and reproducibility in data manipulation and visualization
- Critical for urban mobility and transportation research, with spatial relevance
- Large transportation datasets are becoming increasingly common

Recommended readings

- Engel (2023) Introduction to R.
- Wickham et al. (2017) R for Data Science.
- Lovelace et al. (2024) Geocomputation with R.

3. Software

In this chapter we will guide you through the installation of R, Positron/RStudio IDE and the packages you will need for this course.

R and **Positron**/**RStudio**[1] are separate downloads.

3.1. R

You will need **R** installed on your computer. **R stats** (how it is also known) is a programming language and free software environment for statistical computing and graphics supported by the R Foundation for Statistical Computing.

The download links live at The Comprehensive R Archive Network (aka CRAN). he most recent version is `4.6.1`, but you can use any `>= 4.1.x` if you already have it installed.

3.1.1. Windows

Download R for Windows and run the executable file.

> Important
>
> You may also need to install Rtools, which is a collection of tools necessary to build R packages in Windows.

3.1.2. Mac

Download R for MacOX. You will have to choose between the *arm64* or the *x86-64* version.

Download the `.pkg` file and install it as usual.

[1] We will use Positron. Although if you prefer RStudio or already use other IDE such as VScode, that's also fine.

3.1.3. Ubuntu

> ! These are instructions for Ubuntu. If you use other linux distribution, please follow the instructions on The Comprehensive R Archive Network - CRAN.

You can look for R in the Ubuntu **Software Center** or install it via the terminal:

```
# sudo apt update && sudo apt upgrade -y
sudo apt install r-base
```

Or, if you prefer, you can install the latest version of R from CRAN:

```
# update indices
sudo apt update -qq
# install two helper packages we need
sudo apt install --no-install-recommends software-properties-common
    dirmngr
# add the signing key (by Michael Rutter) for these repos
wget -qO- https://cloud.r-project.org/bin/linux/ubuntu/marutter_pubkey.asc
    | sudo tee -a /etc/apt/trusted.gpg.d/cran_ubuntu_key.asc
# add the R 4.0 repo from CRAN
sudo add-apt-repository "deb https://cloud.r-project.org/bin/linux/ubuntu
    $(lsb_release -cs)-cran40/"
```

> ⚠ **Using an Ubuntu based distribution, but not pure Ubuntu?**
>
> If you are using an Ubuntu based Linux distribution, such as Linux Mint, the last command from the previous script will set the CRAN repository to an invalid address, causing the next step to throw an error when trying to fetch and install R.
> To fix it, run the commands below.
>
> ```
> # Remove invalid repository defined before
> sudo add-apt-repository --remove "deb
>     https://cloud.r-project.org/bin/linux/ubuntu $(lsb_release
>     -cs)-cran40/"
> # Set repository to fixed address, considering Ubuntu version
>     (upstream) instead of your Ubuntu flavour one (which is invalid
>     for CRAN)
> sudo add-apt-repository "deb
>     https://cloud.r-project.org/bin/linux/ubuntu $(source
>     /etc/upstream-release/lsb-release && echo
>     $DISTRIB_CODENAME)-cran40/"
> ```

Then run:

```
sudo apt install r-base r-base-core r-recommended r-base-dev
```

Optional

To keep up-to-date R version and packages, you can follow the instructions at r2u.

After this installation, you don't need to open R base. Please proceed to install the IDE.

3.2. IDE

An IDE (Integrated Development Environment) is a software that provides tools to write, test and execute code in a single graphical interface. It makes the process of programming easier and more accessible to users less proeficient in coding.

Below are tutorials to install two alternatives that support R. Both include a console, syntax-highlighting editor that supports direct code execution, as well as tools for plotting, history, debugging and workspace management. Positron has a more modern UI, but RStudio is more established due to its longevity. You can choose either.

3.2.1. Positron

Positron is available for free download from Posit Positron.

3.2.2. Windows

Download Positron for Windows and run the executable file.

User or system install?

You can choose either. User install will keep Positron available only for your user, while System install will make it available to any user on your computer.

Is Positron failing to start R session?

If after successfully installing Positron you open it and face an error on the R console, installing Microsoft Visual C++ Redistributable can resolve missing dependencies that might be causing the problem.
To install it, open this link and download the `.exe` file in the **Link** column that suits your system architecture. Click on the file and run the instalation procedure. Restart Positron. The issue should be gone.

3.2.3. MacOS

Download Positron for macOS file and install it as usual.

3.2.4. Ubuntu

Download Positron `.deb` file and install it as usual.

3.2.5. RStudio

RStudio is available for free download from Posit RStudio.

3.2.6. Windows

Download RStudio IDE `.exe` file and run the executable file.

3.2.7. MacOS

Download RStudio IDE `.dmg` file and install it as usual.

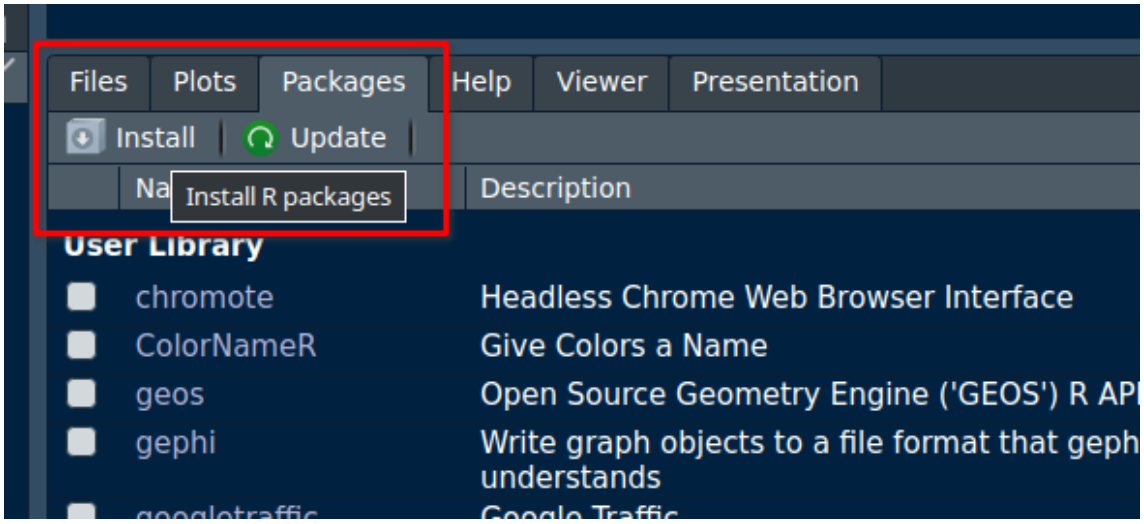
3.2.8. Ubuntu

Download RStudio IDE `.deb` file and install it as usual.

3.3. R packages

You will need to install some packages to work with the data and scripts in this course.

You can install them in RStudio by searching for them in the **Packages** tab:



or by running the following code in the console:

```
install.packages("tidyverse")
install.packages("readxl")

install.packages("sf")
install.packages("mapview")
install.packages("rmarkdown")
install.packages("centr")
install.packages("od")
install.packages("openrouteservice")

install.packages(c("remotes", "devtools", "usethis")) # optional
install.packages("osmextract") # optional
install.packages("stplanr") # optional
```

Facing problems installing packages on Linux?

It is possible that the instalation of the packages above fails due to missing dependencies. Unlike Windows or Mac, Linux sometimes tries to "build" packages from scratch, which causes errors if certain system tools are missing. To avoid this, follow the steps below to tell R to download ready-to-use packages instead of trying to build them.

1. On the **Terminal** (next to the console, at the bottom of your screen, there is a tab called **Terminal**), run the command below to get your OS `codename`:

```
# If you are using an Ubuntu original distro, get the value at
    'codename'
lsb_release -a
# If you use other Ubuntu based distro (such as Linux Mint), get
    the value returned by the command below
source /etc/upstream-release/lsb-release && echo $DISTRIB_CODENAME
```

2. Back on the **Console**, run the following R code, making sure to replace 'resolute' with the codename retrieved in the previous step.

```
# Use pre-compiled binary packages to avoid errors due to missing
    dependencies
options(
  repos = c(CRAN =
      "https://packagemanager.posit.co/cran/__linux__/resolute/latest"),
  HTTPUserAgent = sprintf("R/%s R (%s)", getRversion(),
      paste(getRversion(), R.version["platform"],
      R.version["arch"], R.version["os"]))
)

# Disable bspm interception (use pre-compiled Linux binaries
    directly from Posit without needing apt or system privileges)
# install.packages(c("bspm"))
bspm::disable()
```

3. All set! Try to install the packages again. It should run smoothly now. :)

3.4. Cheat sheets

This is the only cheating allowed at this course.

Cheath seets are a concise set of notes used for quick reference. Posit, the open-source data

science software corporation responsible for RStudio/Positron IDEs, also makes available several cheat sheets that provide a comprehensive introduction to many of R softwares and packages.

You can start by taking a look at the ones for Positron or RStudio.

Refer to rstudio.github.io/cheatsheets for the full cheat sheet repository.

Part I.

Data and Models

4. R basics

In this chapter we will introduce to the R basics and some exercises to get familiar to how R works.

4.1. Math operations

4.1.1. Sum

```
1+1
```

```
[1] 2
```

4.1.2. Subtraction

```
5-2
```

```
[1] 3
```

4.1.3. Multiplication

```
2*2
```

```
[1] 4
```

4.1.4. Division

```
8/2
```

```
[1] 4
```

4.1.5. Round the number

```
round(3.14)
```

```
[1] 3
```

```
round(3.14, 1) # The "1" indicates to round it up to 1 decimal digit.
```

```
[1] 3.1
```

You can use help `?round` in the console to see the description of the function, and the default arguments.

4.2. Basics

4.2.1. Combinations

```
c(1, 2, 3)
```

```
[1] 1 2 3
```

```
c(1:3) # The ":" indicates a range between the first and second numbers.
```

```
[1] 1 2 3
```

Try it yourself

Try to write a combination with the numbers 10, 11, 56, 57, 58.

4.2.2. Create a comment with `ctrl + shift + c`

```
# Comments help you organize your code.
# A line with a comment will not be executed.
```

4.2.3. Create a table

A simple table with the number of trips by *car*, *PT*, *walking*, and *cycling* in a hypothetical street segment at a certain period.

Define variables

```
modes <- c("car", "PT", "walking", "cycling") # you can use "=" or "<-"
trips = c(200, 50, 300, 150) # key sensitive (uppercase and lowercase
    letters are different)
```

Join the variables to create a table

```
table_example = data.frame(modes, trips)
```

Take a look at the table

Visualize the table by clicking on the "Data" in the "Environment" page or use :

```
View(table_example)
```

Look at the first row

```
table_example[1,] #rows and columns start from 1 in R, differently from
    Python which starts from 0.
```

```
  modes trips
1   car   200
```

Look at first row and column

```
table_example[1,1]
```

```
[1] "car"
```

```
table_example[1,4]
```

```
NULL
```

4.3. Practical exercise

Dataset: the number of trips between all municipalities in the Lisbon Metropolitan Area, Portugal (INE 2018).

4.3.1. Import dataset

Download the dataset TRIPSmode_mun.Rds and save it in the folder `data`, under your project's root folder.

To import it, you can click directly in the file under the "Files" pan, or:

```
data = readRDS("data/TRIPSmode_mun.Rds")
```

> After you type " you can use `tab` to navigate between folders and files and `enter` to autocomplete.

4.3.2. Take a first look at the data

Summary statistics

```
summary(data)
```

```
    Origin_mun   Destination_mun     Total              Walk
 Length   :315   Length   :315    Min.   :     7   Min.   :     0
 N.unique : 18   N.unique : 18    1st Qu.:   330   1st Qu.:     0
 N.blank  :  0   N.blank  :  0    Median :  1090   Median :     0
 Min.nchar:  5   Min.nchar:  5    Mean   : 16825   Mean   :  4033
 Max.nchar: 19   Max.nchar: 19    3rd Qu.:  5374   3rd Qu.:     0
                                  Max.   :875144   Max.   :306289
     Bike              Car             PTransit            Other
 Min.   :   0.00   Min.   :     0   Min.   :     0.0   Min.   :    0.0
 1st Qu.:   0.00   1st Qu.:   263   1st Qu.:     5.0   1st Qu.:    0.0
 Median :   0.00   Median :   913   Median :   134.0   Median :    0.0
 Mean   :  80.19   Mean   :  9956   Mean   :  2602.6   Mean   :  152.4
 3rd Qu.:   0.00   3rd Qu.:  4408   3rd Qu.:   975.5   3rd Qu.:   62.5
 Max.   :5362.00   Max.   :349815   Max.   :202428.0   Max.   :11647.0
```

Check the structure of the data

```
str(data)
```

```
Classes 'tbl_df', 'tbl' and 'data.frame':   315 obs. of  8 variables:
 $ Origin_mun     : chr  "Alcochete" "Alcochete" "Alcochete" "Alcochete" ...
 $ Destination_mun: chr  "Alcochete" "Almada" "Amadora" "Barreiro" ...
 $ Total          : num  20478 567 188 867 114 ...
 $ Walk           : num  6833 0 0 0 0 ...
 $ Bike           : num  320 0 0 0 0 0 0 0 91 0 ...
```

```
 $ Car            : num  12484 353 107 861 114 ...
 $ PTransit       : num  833 0 81 5 0 ...
 $ Other          : num  7 214 0 0 0 0 0 0 0 0 ...
```

Check the number of rows (observations) and columns (variables)

```
nrow(data)
```

```
[1] 315
```

```
ncol(data)
```

```
[1] 8
```

Open the dataset

```
View(data)
```

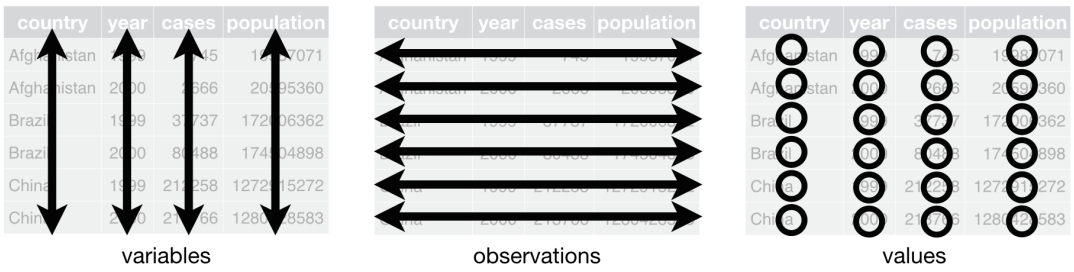


Figure 4.1.: The following three rules make a dataset tidy: variables are columns, observations are rows, and values are cells. [@wickham17r]

4.3.3. Explore the data

Check the total number of trips

Use `$` to select a variable of the data

```
sum(data$Total)
```

```
[1] 5299853
```

Percentage of car trips related to the total

```
sum(data$Car)/sum(data$Total) * 100
```

```
[1] 59.17638
```

Percentage of active trips related to the total

```
(sum(data$Walk) + sum(data$Bike)) / sum(data$Total) * 100
```

```
[1] 24.44883
```

4.3.4. Modify original data

Create a column with the sum of the number of trips for active modes

```
data$Active = data$Walk + data$Bike
```

Filter by condition (create new tables)

Filter trips only with origin from Lisbon

```
data_Lisbon = data[data$Origin == "Lisboa",]
```

Filter trips with origin **different** from Lisbon

```
data_out_Lisbon = data[data$Origin != "Lisboa",]
```

Filter trips with origin **and** destination in Lisbon

```
data_in_Out_Lisbon = data[data$Origin == "Lisboa" & data$Destination ==
    "Lisboa",]
```

Remove the first column

```
data = data[ ,-1] #first column
```

Create a table only with origin, destination and walking trips

There are many ways to do the same operation.

```r
names(data)
```

```
[1] "Destination_mun" "Total"           "Walk"            "Bike"
[5] "Car"             "PTransit"        "Other"           "Active"
```

```r
data_walk2 = data[ ,c(1,2,4)]
```

```r
data_walk3 = data[ ,-c(3,5:9)]
```

4.3.5. Export data

Save data in **.csv** and **.Rds**

```r
write.csv(data, 'data/dataset.csv', row.names = FALSE)
saveRDS(data, 'data/dataset.Rds') #Choose a different file.
```

4.3.6. Import data

```r
csv_file = read.csv("data/dataset.csv")
rds_file = readRDS("data/dataset.Rds")
```

5. Data manipulation

06/10/2026 class materials

Download class_2.zip, unzip it and open it on your IDE.

In this chapter we will use some very useful `dplyr` functions to handle and manipulate data.

You can load the `dplyr` package directly, or load the entire tidy universe (`tidyverse`).

```
# library(tidyverse)
library(dplyr)
```

Using the same dataset as in R basics but with slightly differences[1].

We will do the same operations but in a simplified way.

Download the dataset TRIPSorigin.Rds and save it in the folder `data`, under your project's root folder.

```
TRIPS <- readRDS("data/TRIPSorigin.Rds")
```

Important

Note that it is very important to understand the R basics, that's why we started from there, even if the following functions will provide the same results.

You don't need to know everything! And you don't need to know by heart. The following functions are the ones you will probably use most of the time to handle data.

There are several ways to reach the same solution. Here we present only one of them.

[1] This dataset includes the number of trips with origin in each neighborhood, divided by mode of transport, and inter or intra municipal trips.

5.1. Select variables

Have a look at your dataset. You can open using `View()`, look at the information at the "Environment" panel, or even print the same information using `glimpse()`

```
glimpse(TRIPS)
```

We will create a new dataset with *Origin*, *Walk, Bike* and *Total*. This time we will use the `select()` function.

```
TRIPS_new <- select(TRIPS, Origin, Walk, Bike, Total) # the first argument
    is the dataset
```

The first argument, as usually in R, is the dataset, and the remaining ones are the columns to select.

With most of the `dplyr` functions you don't need to refer to `data$...` you can simply type the variable names (and even without the `"..."`!). This makes coding in R simpler :)

You can also remove columns that you don't need.

```
TRIPS_new <- select(TRIPS_new, -Total) # dropping the Total column
```

5.1.1. Using pipes!

Now, let's introduce pipes. Pipes are a rule as: "**With this, do this.**"

This is useful to skip the first argument of the functions (usually the dataset to apply the function).

Applying a pipe to the `select()` function, we can write as:

```
TRIPS_new <- TRIPS |> select(Origin, Walk, Bike, Total)
```

Two things to **note**:

1. The pipe symbol can be written as `|>` or `%>%`. [2] To write it you may also use the `ctrl+shift+m` shortcut.
2. After typing `select(` you can press `tab` and the list of available variables of that dataset will show up! `Enter` to select. With this you prevent typo errors.

[2] You can change this in RStudio > Tools > Global Options > Code.

5.2. Filter observations

You can filter observations based on a condition using the `filter()` function.

```
TRIPS2 <- TRIPS[TRIPS$Total > 25000, ] # using r-base, you cant forget the
    comma
TRIPS2 <- TRIPS2 |> filter(Total > 25000) # using dplyr, it's easier
```

You can have other conditions inside the condition.

```
summary(TRIPS$Total)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    361    5918   17474   22457   33378  112186
```

```
TRIPS3 <- TRIPS |> filter(Total > median(Total))
```

Other filter conditions:

- `==` equal, `!=` different
- `<` smaller, `>` greater, `<=` smaller or equal, `>=` greater or equal
- `&` and, `|` or
- `is.na`, `!is.na` is not NA
- `%in%`, `!%in%` not in

5.3. Create new variables

You can also try again to create a variable of Car percentage using pipes! To create a new variable or change an existing one (overwriting), you can use the `mutate()` function.

```
TRIPS$Car_perc <- TRIPS$Car / TRIPS$Total * 100 # using r-base

TRIPS <- TRIPS |> mutate(Car_perc = Car / Total * 100) # using dplyr
```

5.4. Change data type

Data can be in different formats. For example, the variable *Origin* is a character, but we can convert it to a numeric variable.

```
class(TRIPS$Origin)
```

```
[1] "character"
```

```
TRIPS <- TRIPS |>
    mutate(Origin_num = as.integer(Origin)) # you can use as.numeric() as
        well
class(TRIPS$Origin_num)
```

```
[1] "integer"
```

Most used data types are:

- integer (`int`)
- numeric (`num`)
- character (`chr`)
- logical (`logical`)
- date (`Date`)
- factor (`factor`)

5.4.1. Factors

Factors are useful to deal with categorical data. You can convert a character to a factor using `as.factor()`, and also use labels and levels for categorical ordinal data.

We can change the `Lisbon` variable to a factor, and `Internal` too.

```
TRIPS <- TRIPS |>
    mutate(
        Lisbon_factor = factor(Lisbon, labels = c("No", "Yes")),
        Internal_factor = factor(Internal, labels = c("Inter", "Intra"))
    )
```

But how do we know which levels come first? A simple way is to use `table()` or `unique()` functions.

```
unique(TRIPS$Lisbon) # this will show all the different values
```

```
[1] 0 1
```

```
table(TRIPS$Lisbon) # this will show the frequency of each value
```

```
  0   1
188  48
```

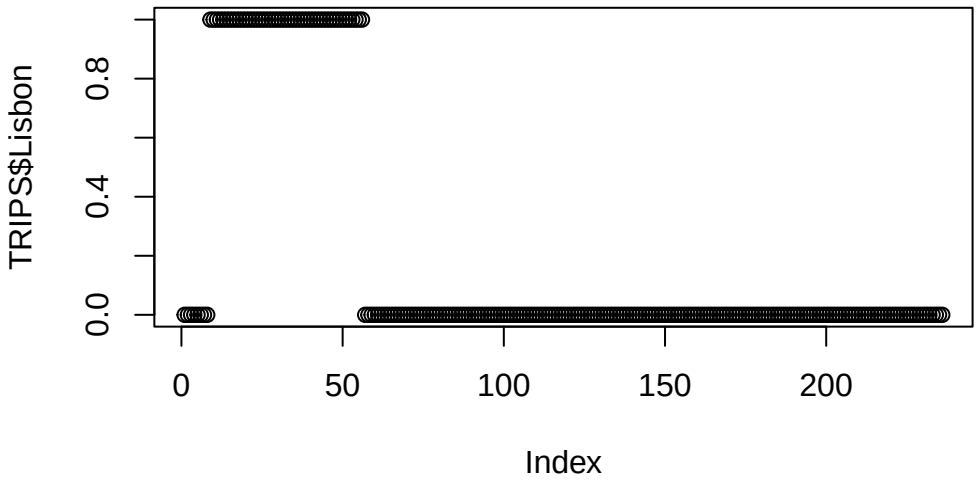
```
table(TRIPS$Lisbon_factor)
```

```
 No Yes
188  48
```

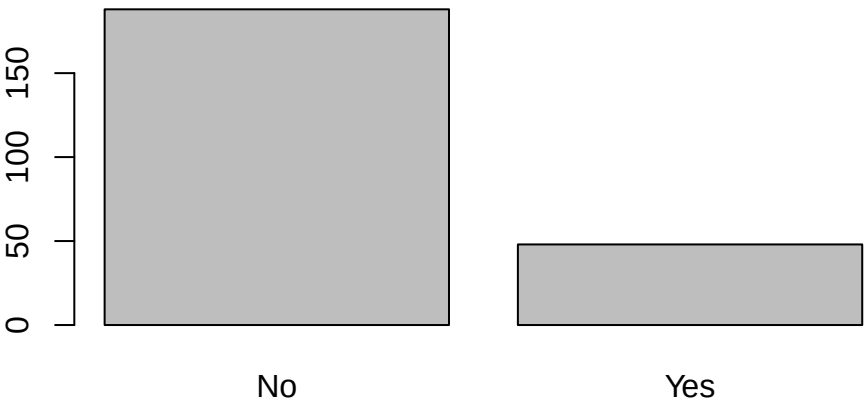
The first number to appear is the first level, and so on.

You can see the difference between using a continuous variable (in this case `Lisbon`‘ has 0 and 1) and a categorical variable (`Lisbon_factor`).

```
plot(TRIPS$Lisbon) # the values range between 0 and 1
```




```
plot(TRIPS$Lisbon_factor) # the values are categorical and labeled with
    Yes/No
```



5.5. Join data tables

When having relational tables - *i.e.* with a common identifier - it is useful to be able to join them in a very efficient way.

`left_join` is a function that joins two tables **by a common column**. The **first table is the one that will be kept**, and the **second one will be joined to** it. How `left_join` works:

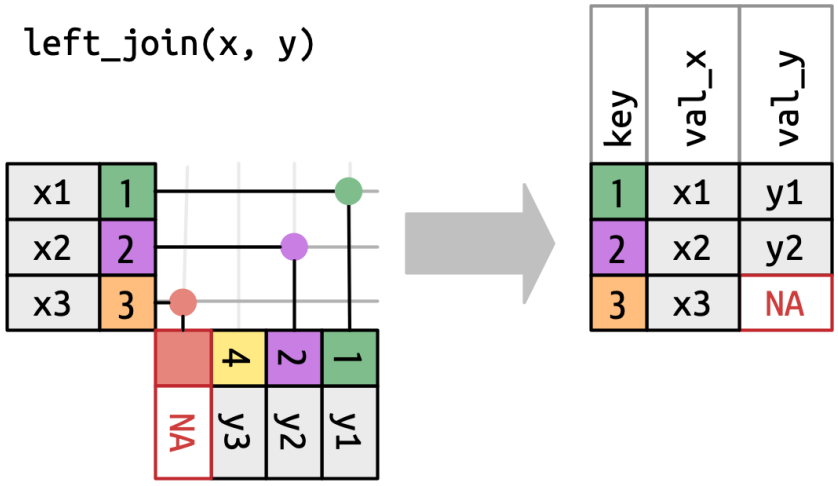


Figure 5.1.: A visual representation of the left join where every row in x appears in the output.Source: R for Data Science.

Let's **join the municipalities** to this table with a supporting table that includes all the **relation** between neighbourhoods and municipalities, and the respective names and codes.

Download the dataset Municipalities_names.Rds and save it in the folder `data`, under your project's root folder.

```
Municipalities <- readRDS("data/Municipalities_names.Rds")
```

```
head(TRIPS)
```

```
# A tibble: 6 x 13
  Origin Total  Walk  Bike   Car PTransit Other Internal Lisbon Car_perc
  <chr>  <dbl> <dbl> <dbl> <dbl>    <dbl> <dbl>    <dbl>  <dbl>    <dbl>
1 110501 35539 11325  1309 21446     1460     0        0      0     60.3
2 110501 47602  3502   416 37727     5519   437        1      0     79.3
3 110506 37183 12645    40 22379     2057    63        0      0     60.2
4 110506 42313  1418   163 37337     3285   106        1      0     88.2
5 110507 30725  9389  1481 19654      201     0        0      0     64.0
6 110507 54586  2630   168 44611     6963   215        1      0     81.7
```

```
# i 3 more variables: Origin_num <int>, Lisbon_factor <fct>,
#   Internal_factor <fct>
```

```
tail(Municipalities)
```

```
    Mun_code Neighborhood_code        Municipality
113     1109            110913               Mafra
114     1114            111409 Vila Franca de Xira
115     1109            110918               Mafra
116     1109            110904               Mafra
117     1502            150202           Alcochete
118     1109            110911               Mafra
                                            Neighborhood
113                                        Santo Isidoro
114                                     Vila Franca de Xira
115 União das freguesias de Azueira e Sobral da Abelheira
116                                          Encarnação
117                                             Samouco
118                                           Milharado
```

We can see that we have a common variable: `Origin` in `TRIPS` and `Neighborhood_code` in `Municipalities`.

To join these two tables we need to specify the common variable in each table, using the `by` argument.

```
TRIPSjoin <- TRIPS |> left_join(Municipalities, by = c("Origin" =
    "Neighborhood_code"))
```

If you prefer, you can mutate or rename a variable so both tables have the same name. When **both tables have the same name**, you don't need to specify the `by` argument.

```
Municipalities <- Municipalities |> rename(Origin = "Neighborhood_code") #
    change name
TRIPSjoin <- TRIPS |> left_join(Municipalities) # automatic detects common
    variable
```

As you can see, both tables don't need to be the same length. The `left_join` function will keep all the observations from the first table, and join the second table to it. If there is no match, the variables from the second table will be filled with `NA`.

5.6. group_by and summarize

We have a very large table with all the neighbourhoods and their respective municipalities. We want to know the total number of trips with origin in each municipality.

To make it easier to understand, let's keep only the variables we need.

```
TRIPSredux <- TRIPSjoin |> select(Origin, Municipality, Internal, Car,
    Total)
head(TRIPSredux)
```

```
# A tibble: 6 x 5
  Origin Municipality Internal   Car Total
  <chr>  <chr>           <dbl> <dbl> <dbl>
1 110501 Cascais             0 21446 35539
2 110501 Cascais             1 37727 47602
3 110506 Cascais             0 22379 37183
4 110506 Cascais             1 37337 42313
5 110507 Cascais             0 19654 30725
6 110507 Cascais             1 44611 54586
```

We can group this table by the `Municipality` variable and summarize the number of trips with origin in each municipality.

```
TRIPSsum <- TRIPSredux |>
    group_by(Municipality) |> # you won't notice any chagne with only this
    summarize(Total = sum(Total))
head(TRIPSsum)
```

```
# A tibble: 6 x 2
  Municipality   Total
  <chr>          <dbl>
1 Alcochete      36789
2 Almada        289834
3 Amadora       344552
4 Barreiro      133658
5 Cascais       373579
6 Lisboa       1365111
```

We summed the total number of trips in each municipality.

If we want to group by more than one variable, we can add more `group_by()` functions.

```
TRIPSsum2 <- TRIPSredux |>
    group_by(Municipality, Internal) |>
    summarize(
        Total = sum(Total),
        Car = sum(Car)
    )
head(TRIPSsum2)
```

```
# A tibble: 6 x 4
# Groups:   Municipality [3]
  Municipality Internal  Total    Car
  <chr>           <dbl>  <dbl>  <dbl>
1 Alcochete           0  16954   9839
2 Alcochete           1  19835  15632
3 Almada              0 105841  49012
4 Almada              1 183993 125091
5 Amadora             0 117727  33818
6 Amadora             1 226825 142386
```

We summed the total number of trips and car trips in each municipality, **separated by** inter and intra municipal trips.

> It is a good practice to use the `ungroup()` function after the `group_by()` function. This will remove the grouping. If you don't do this, the grouping will be kept and you may have unexpected results in the next time you use that dataset.

5.7. Arrange data

You can **sort** a dataset by one or more variables.

For instance, `arrange()` by Total trips, ascending or descending order.

```
TRIPS2 <- TRIPSsum2 |> arrange(Total)
TRIPS2 <- TRIPSsum2 |> arrange(-Total) # descending

TRIPS2 <- TRIPSsum2 |> arrange(Municipality) # alphabetic

TRIPS4 <- TRIPS |> arrange(Lisbon_factor, Total) # more than one variable
```

This is not the same as opening the view table and click on the arrows. When you do that, the order is not saved in the dataset. If you want to save the order, you need to use the `arrange()` function.

5.8. All together now!

This is the pipes magic. It takes the last result and applies the next function to it. "With this, do this.". You can chain as many functions as you want.

```
TRIPS_pipes <- TRIPS |>
    select(Origin, Internal, Car, Total) |>
    mutate(Origin_num = as.integer(Origin)) |>
    mutate(Internal_factor = factor(Internal, labels = c("Inter",
        "Intra"))) |>
    filter(Internal_factor == "Inter") |>
    left_join(Municipalities) |>
    group_by(Municipality) |>
    summarize(
        Total = sum(Total),
        Car = sum(Car),
        Car_perc = Car / Total * 100
    ) |>
    ungroup() |>
    arrange(desc(Car_perc))
```

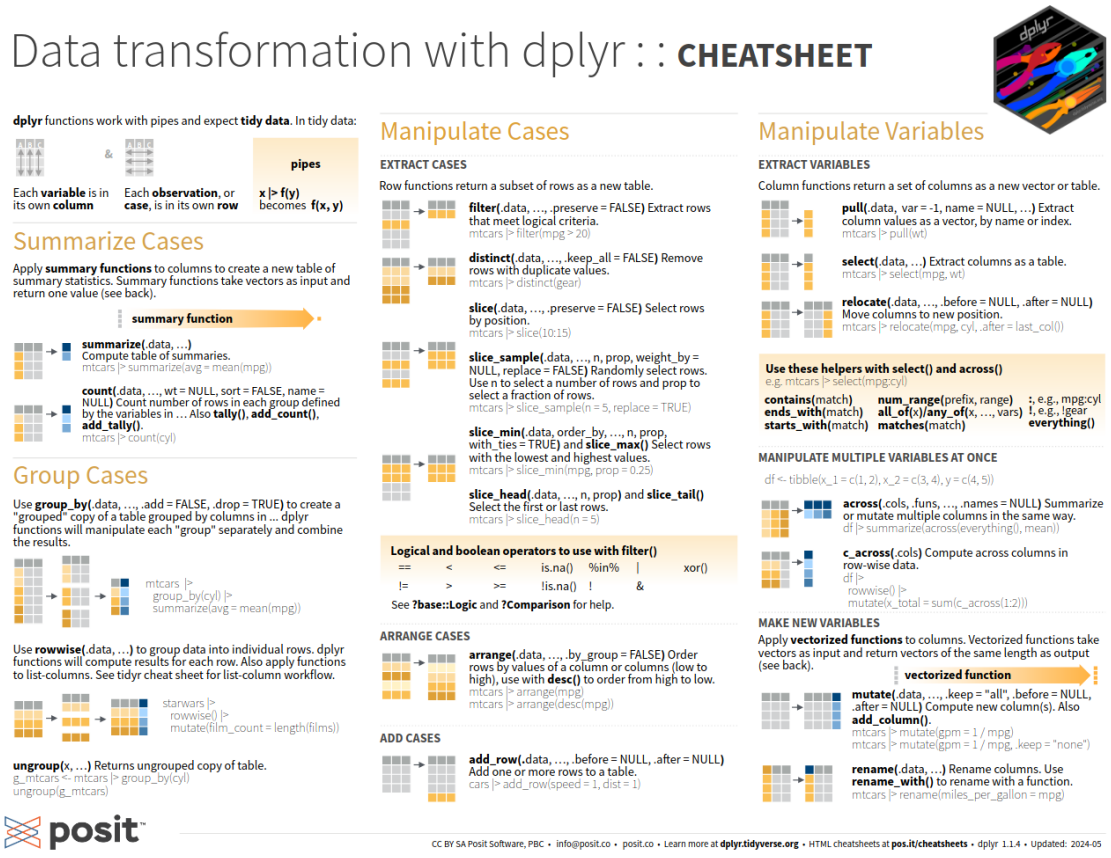
With this code we will have a table with the total number of intercity trips, by municipality, with their names instead of codes, arranged by the percentage of car trips.

```
TRIPS_pipes
```

```
# A tibble: 18 x 4
   Municipality         Total    Car Car_perc
   <chr>                <dbl>  <dbl>    <dbl>
 1 Mafra                65811  46329     70.4
 2 Sesimbra             49370  31975     64.8
 3 Cascais             161194  96523     59.9
 4 Palmela              66428  39688     59.7
 5 Alcochete            16954   9839     58.0
 6 Setúbal             129059  70318     54.5
 7 Montijo              57164  30900     54.1
 8 Seixal              120747  63070     52.2
 9 Sintra              237445 123408     52.0
10 Oeiras              134862  66972     49.7
11 Almada              105841  49012     46.3
12 Loures              132310  60478     45.7
13 Barreiro             52962  24160     45.6
14 Odivelas             93709  39151     41.8
15 Vila Franca de Xira 115152  47201     41.0
16 Moita                51040  17394     34.1
17 Amadora             117727  33818     28.7
18 Lisboa              280079  69038     24.6
```

5.9. Other dplyr functions

You can explore other `dplyr` functions and variations to manipulate data in the **dplyr cheat sheet**:



Take a particular attention to `pivot_wider` and `pivot_longer` (`tidyr` package) to transform **OD matrices** in **wide** and **long** formats.

Table 5.1.: OD matrix in long format

Origins	Destinations	Trips
A	B	20
A	C	45
B	A	10
C	C	5
C	A	30

Table 5.2.: OD matrix in wide format

Trips	A	B	C
A	NA	20	45

Trips	A	B	C
B	10	NA	NA
C	30	NA	5

6. Exploratory Data Analysis

6.1. Intro

This chapter will show you how to use visualization and transformation to explore your data in a systematic way, a task that statisticians call exploratory data analysis, or **EDA** for short. EDA is an iterative cycle, where you should feel free to "fall in love" with your data.

You:

1. Generate questions about your data.
2. Search for answers by visualizing, transforming, and modelling your data.
3. Use what you learn to refine your questions and/or generate new questions.

   Note: EDA is not a formal process with a strict set of rules!

See Wickham et al. (2017) for more, in particular the EDA chapter.

> **Do it yourself with R**
>
> Copy the script ExploratoryDataAnalysis.R and paste it in your session.
> Run each line using `CTRL` + `ENTER`

6.2. Load packages

```
library(tidyverse) # Pack of most used libraries for data science
library(skimr) # summary of the data
library(DataExplorer) # exploratory data analysis
library(corrplot) # correlation plots
```

6.3. Dataset

The database used in this example is a treated database from the Mobility Survey for the metropolitan areas of Lisbon and Porto in 2018 (INE 2018). We will only focus on trips within the metropolitan area of Lisbon.

Included **variables**:

- `Origin_dicofre16` - Code of Freguesia (district) as set by INE after 2016 (Distrito + Concelho + Freguesia), for trip origin
- `Total` - number of trips with origin at each district
- `Walk` - number of walking trips
- `Bike` - number of bike trips
- `Car` - number of car trips. Includes taxi and motorcycle.
- `PTransit` - number of Public Transit trips
- `Other` - number of other trips (truck, van, tractor, aviation)
- `Distance` - average trip distance (km)
- `Duration` - average trip duration (minutes)
- `Car_perc` - percentage of car trips
- `N_INDIVIDUOS` - number of residents (INE 2022)
- `Male_perc` - percentage of male residents (INE 2022)
- `IncomeHH` - average household income
- `Nvehicles` - average number of car/motorcycle vehicles in the household
- `DrivingLic` - percentage of car driving licence holders
- `CarParkFree_Work` - percentage of respondents with free car parking at the work location
- `PTpass` - percentage of public transit monthly pass holders
- `internal` - binary variable (factor). `Yes`: trip with same TAZ origin and destination, `No`: trips with different destination
- `Lisboa` - binary variable (factor). `Yes`: the district is part of Lisbon municipality, `No`: otherwise
- `Area_km2` - area of in `Origin_dicofre16`, in km^2

6.3.1. Import dataset

Download the dataset IMOBmodel.Rds and save it in the folder `data`, under your project's root folder.

```
data = readRDS("data/IMOBmodel.Rds")
```

Take a look at the dataset

```
View(data) # open in table
glimpse(data) # glimpse of the dataset
```

```
Rows: 236
Columns: 20
$ Origin_dicofre16 <chr> "110501", "110501", "110506", "110506", "110507", "11~
$ Total            <dbl> 35539, 47602, 37183, 42313, 30725, 54586, 57747, 6788~
$ Walk             <dbl> 11325, 3502, 12645, 1418, 9389, 2630, 20423, 3573, 62~
$ Bike             <dbl> 1309, 416, 40, 163, 1481, 168, 1406, 809, 13, 37, 3, ~
$ Car              <dbl> 21446, 37727, 22379, 37337, 19654, 44611, 33044, 5225~
$ PTransit         <dbl> 1460, 5519, 2057, 3285, 201, 6963, 2477, 10534, 110, ~
```

```
$ Other            <dbl> 0, 437, 63, 106, 0, 215, 396, 717, 2, 169, 3, 824, 0,~
$ Distance         <dbl> 11.779, 11.779, 9.868, 9.868, 9.600, 9.600, 12.875, 1~
$ Duration         <dbl> 23.96, 23.96, 22.78, 22.78, 23.39, 23.39, 26.89, 26.8~
$ Car_perc         <dbl> 60.34497, 79.25507, 60.18611, 88.24002, 63.96745, 81.~
$ N_INDIVIDUOS     <dbl> 44165, 44165, 59238, 59238, 46529, 46529, 64192, 6419~
$ Male_perc        <dbl> 47.66, 47.66, 47.12, 47.12, 45.54, 45.54, 46.14, 46.1~
$ IncomeHH         <dbl> 2204.32, 2204.32, 1896.05, 1896.05, 2430.43, 2430.43,~
$ Nvehicles        <dbl> 1.86, 1.86, 1.73, 1.73, 1.60, 1.60, 1.64, 1.64, 1.20,~
$ DrivingLic       <dbl> 63.03, 63.03, 65.42, 65.42, 67.05, 67.05, 72.75, 72.7~
$ CarParkFree_Work <dbl> 49.97, 49.97, 57.09, 57.09, 47.43, 47.43, 56.47, 56.4~
$ PTpass           <dbl> 11.14, 11.14, 17.12, 17.12, 18.25, 18.25, 19.73, 19.7~
$ internal         <fct> Yes, No, Yes, No, Yes, No, Yes, No, Yes, No, Yes, No,~
$ Lisboa           <fct> No, No, No, No, No, No, No, No, Yes, Yes, Yes, Yes, Y~
$ Area_km2         <dbl> 39.767414, 39.767414, 20.364310, 20.364310, 8.109861,~
```

```
str(data) # Structure of the dataset
```

```
'data.frame':   236 obs. of  20 variables:
 $ Origin_dicofre16: chr  "110501" "110501" "110506" "110506" ...
 $ Total           : num  35539 47602 37183 42313 30725 ...
 $ Walk            : num  11325 3502 12645 1418 9389 ...
 $ Bike            : num  1309 416 40 163 1481 ...
 $ Car             : num  21446 37727 22379 37337 19654 ...
 $ PTransit        : num  1460 5519 2057 3285 201 ...
 $ Other           : num  0 437 63 106 0 215 396 717 2 169 ...
 $ Distance        : num  11.78 11.78 9.87 9.87 9.6 ...
 $ Duration        : num  24 24 22.8 22.8 23.4 ...
 $ Car_perc        : num  60.3 79.3 60.2 88.2 64 ...
 $ N_INDIVIDUOS    : num  44165 44165 59238 59238 46529 ...
 $ Male_perc       : num  47.7 47.7 47.1 47.1 45.5 ...
 $ IncomeHH        : num  2204 2204 1896 1896 2430 ...
 $ Nvehicles       : num  1.86 1.86 1.73 1.73 1.6 1.6 1.64 1.64 1.2 1.2 ...
 $ DrivingLic      : num  63 63 65.4 65.4 67 ...
 $ CarParkFree_Work: num  50 50 57.1 57.1 47.4 ...
 $ PTpass          : num  11.1 11.1 17.1 17.1 18.2 ...
 $ internal        : Factor w/ 2 levels "Yes","No": 1 2 1 2 1 2 1 2 1 2 ...
 $ Lisboa          : Factor w/ 2 levels "No","Yes": 1 1 1 1 1 1 1 1 2 2 ...
 $ Area_km2        : num  39.77 39.77 20.36 20.36 8.11 ...
```

6.4. Summary statistics

Have an overview of the variables and their statistics' summary.

```
summary(data) # Check the summary statistics
```

```
 Origin_dicofre16     Total             Walk              Bike
Length   :236     Min.   :   361   Min.   :    0.0   Min.   :   0.00
N.unique :118     1st Qu.:  5918   1st Qu.:  763.2   1st Qu.:   0.00
N.blank  :  0     Median : 17474   Median : 3125.0   Median :  13.50
Min.nchar:  6     Mean   : 22457   Mean   : 5383.4   Mean   : 107.03
Max.nchar:  6     3rd Qu.: 33378   3rd Qu.: 8298.5   3rd Qu.:  98.75
                  Max.   :112186   Max.   :32646.0   Max.   :2040.00
      Car            PTransit         Other           Distance
Min.   :    0   Min.   :    0   Min.   :   0.0   Min.   : 6.835
1st Qu.: 3243   1st Qu.:  249   1st Qu.:   2.0   1st Qu.: 9.539
Median : 9008   Median : 1057   Median :  44.0   Median :10.323
Mean   :13289   Mean   : 3474   Mean   : 203.4   Mean   :11.139
3rd Qu.:21249   3rd Qu.: 4853   3rd Qu.: 281.5   3rd Qu.:12.097
Max.   :52631   Max.   :41672   Max.   :2391.0   Max.   :22.660
   Duration        Car_perc      N_INDIVIDUOS     Male_perc
Min.   :16.30   Min.   : 0.00   Min.   : 1566   Min.   :44.61
1st Qu.:23.00   1st Qu.:45.40   1st Qu.:11060   1st Qu.:46.58
Median :24.70   Median :62.62   Median :20855   Median :47.51
Mean   :25.42   Mean   :59.00   Mean   :24324   Mean   :47.54
3rd Qu.:27.73   3rd Qu.:75.72   3rd Qu.:36079   3rd Qu.:48.29
Max.   :37.42   Max.   :99.27   Max.   :68649   Max.   :55.94
   IncomeHH        Nvehicles       DrivingLic    CarParkFree_Work
Min.   : 884.5   Min.   :1.020   Min.   :37.04   Min.   : 5.47
1st Qu.:1417.8   1st Qu.:1.350   1st Qu.:57.67   1st Qu.:40.39
Median :1594.7   Median :1.545   Median :63.00   Median :50.51
Mean   :1732.6   Mean   :1.530   Mean   :62.50   Mean   :49.30
3rd Qu.:1953.5   3rd Qu.:1.670   3rd Qu.:68.84   3rd Qu.:57.92
Max.   :3462.3   Max.   :2.420   Max.   :80.79   Max.   :87.60
    PTpass      internal  Lisboa       Area_km2
Min.   : 0.00   Yes:118   No :188   Min.   :  1.494
1st Qu.:13.41   No :118   Yes: 48   1st Qu.:  5.044
Median :22.71                       Median : 11.598
Mean   :23.82                       Mean   : 25.553
3rd Qu.:32.94                       3rd Qu.: 28.511
Max.   :60.45                       Max.   :282.125
```

```
skim(data) # In a more organized way
```

Table 6.1.: Data summary

Name	data
Number of rows	236
Number of columns	20
Column type frequency:	
character	1
factor	2
numeric	17
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
Origin_dicofre16	0	1	6	6	0	118	0

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
internal	0	1	FALSE	2	Yes: 118, No: 118
Lisboa	0	1	FALSE	2	No: 188, Yes: 48

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
Total	0	1	22457.00	19084.45	361.00	5917.75	17474.00	33377.50	112186.00	
Walk	0	1	5383.44	6224.84	0.00	763.25	3125.00	8298.50	32646.00	
Bike	0	1	107.03	248.65	0.00	0.00	13.50	98.75	2040.00	
Car	0	1	13289.24	12351.61	0.00	3243.00	9008.00	21248.75	52631.00	
PTransit	0	1	3473.79	5467.82	0.00	249.00	1057.00	4853.00	41672.00	
Other	0	1	203.45	336.04	0.00	2.00	44.00	281.50	2391.00	
Distance	0	1	11.14	2.66	6.84	9.54	10.32	12.10	22.66	
Duration	0	1	25.42	3.91	16.30	23.00	24.70	27.73	37.42	

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
Car_perc	0	1	59.00	21.56	0.00	45.40	62.62	75.72	99.27	
N_INDIVIDUOS	0	1	24323.80	16438.04	1566.00	11060.00	20855.00	36079.00	68649.00	
Male_perc	0	1	47.54	1.57	44.61	46.58	47.50	48.29	55.94	
IncomeHH	0	1	1732.55	453.11	884.46	1417.76	1594.73	1953.50	3462.32	
Nvehicles	0	1	1.53	0.24	1.02	1.35	1.54	1.67	2.42	
DrivingLic	0	1	62.50	8.12	37.04	57.67	63.00	68.84	80.79	
CarPark-Free_Work	0	1	49.30	14.30	5.47	40.39	50.51	57.92	87.60	
PTpass	0	1	23.82	12.87	0.00	13.41	22.72	32.94	60.45	
Area_km2	0	1	25.55	42.58	1.49	5.04	11.60	28.51	282.13	

6.5. Missing data

Is there missing data (`NA`)? How many?

```
table(is.na(data))
```

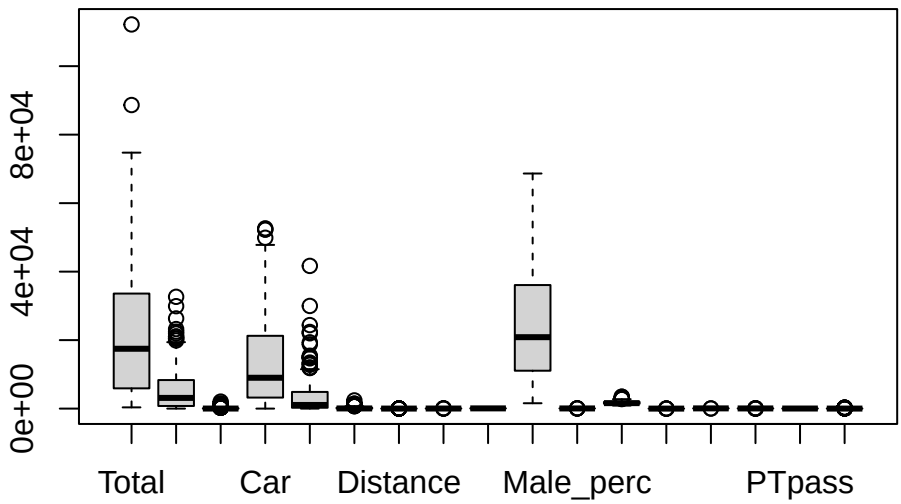
```
FALSE
 4720
```

6.6. Outliers

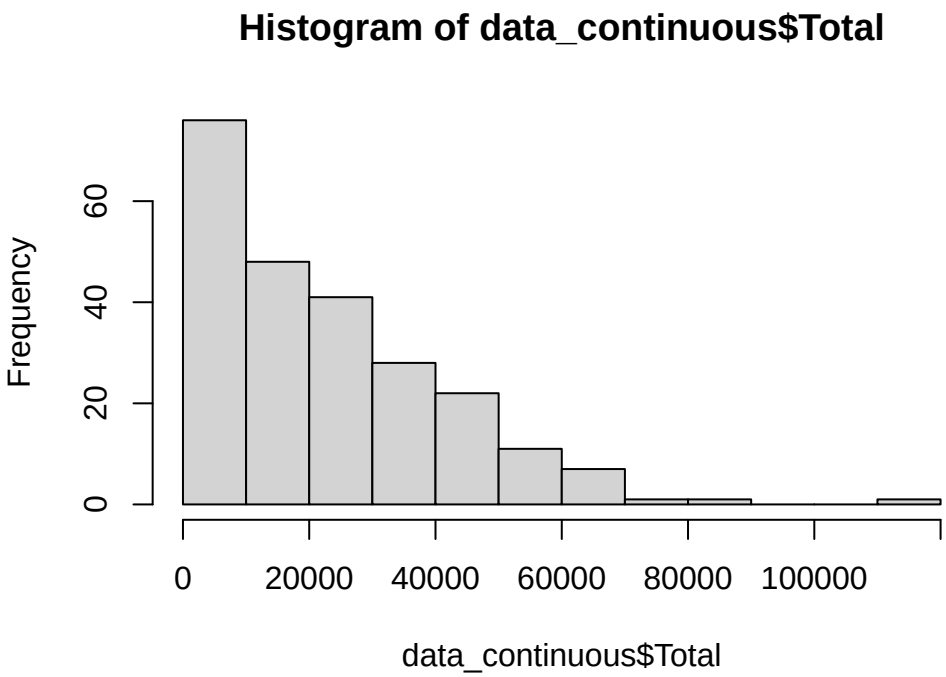
Inspect outliers and distributions

```
boxplot(data) # This does now work if variables are not all continuous
```

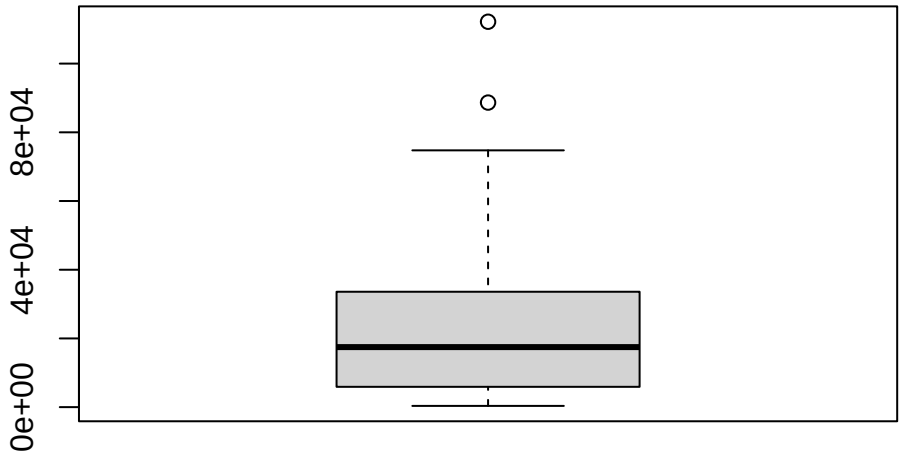
```
data_continuous = data |> select(-Origin_dicofre16, -internal, -Lisboa) #
    Exclude categorical variables
boxplot(data_continuous) # Exclude categorical variables
```



```
hist(data_continuous$Total) # histogram
```




```
boxplot(data_continuous$Total) # outliers detected
```



6.6.1. Treating outliers

Based on the theory, we can create a function to detect the outliers.

```
outlier = function(x) {
  q = quantile(x, probs = c(0.25, 0.75), na.rm = TRUE)  # Q1 and Q3
  caps = quantile(x, probs = c(0.05, 0.95), na.rm = TRUE) # 5th and 95th
    percentile
```

```
  H = 1.5 * IQR(x, na.rm = TRUE) # interquartile range

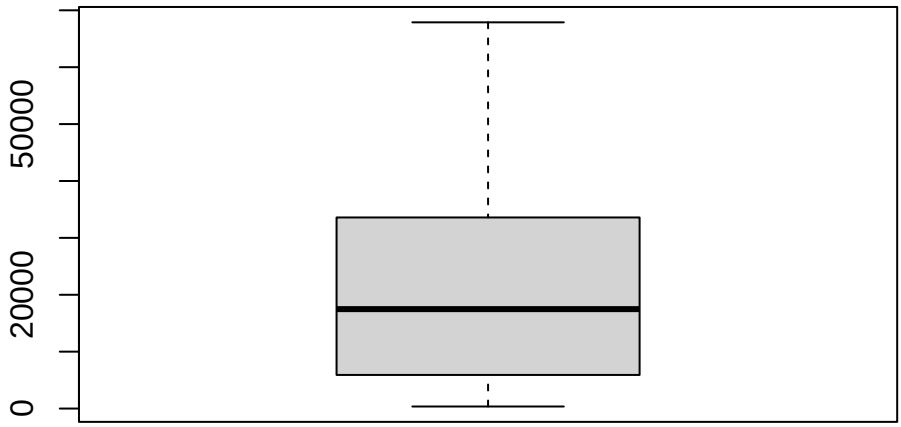
  case_when(
    x < (q[1] - H) ~ caps[1], # replace values that are LESS than
    Q1-1.5*IQR with the P5 value
    x > (q[2] + H) ~ caps[2], # replace values that are MORE than
    Q3+1.5*IQR with the P95 value
    TRUE ~ x # otherwise, return the original value
  )
}
```

Now, use it in a copy of the table

```
data_outliers = data_continuous # duplicate the table
data_outliers$Total = outlier(data_outliers$Total) # Use the function to
    the same variable
```

Take a look again at the boxplot

```
boxplot(data_outliers$Total)
```



Compare the variable with and without the outliers

```
# Mean
mean(data$Total)
```

```
[1] 22457
```

```
mean(data_outliers$Total)
```

```
[1] 22013.51
```

```
# Median
median(data$Total)
```

```
[1] 17474
```

```
median(data_outliers$Total)
```

```
[1] 17474
```

```
# Standard deviation
sd(data$Total)
```

```
[1] 19084.45
```

```
sd(data_outliers$Total)
```

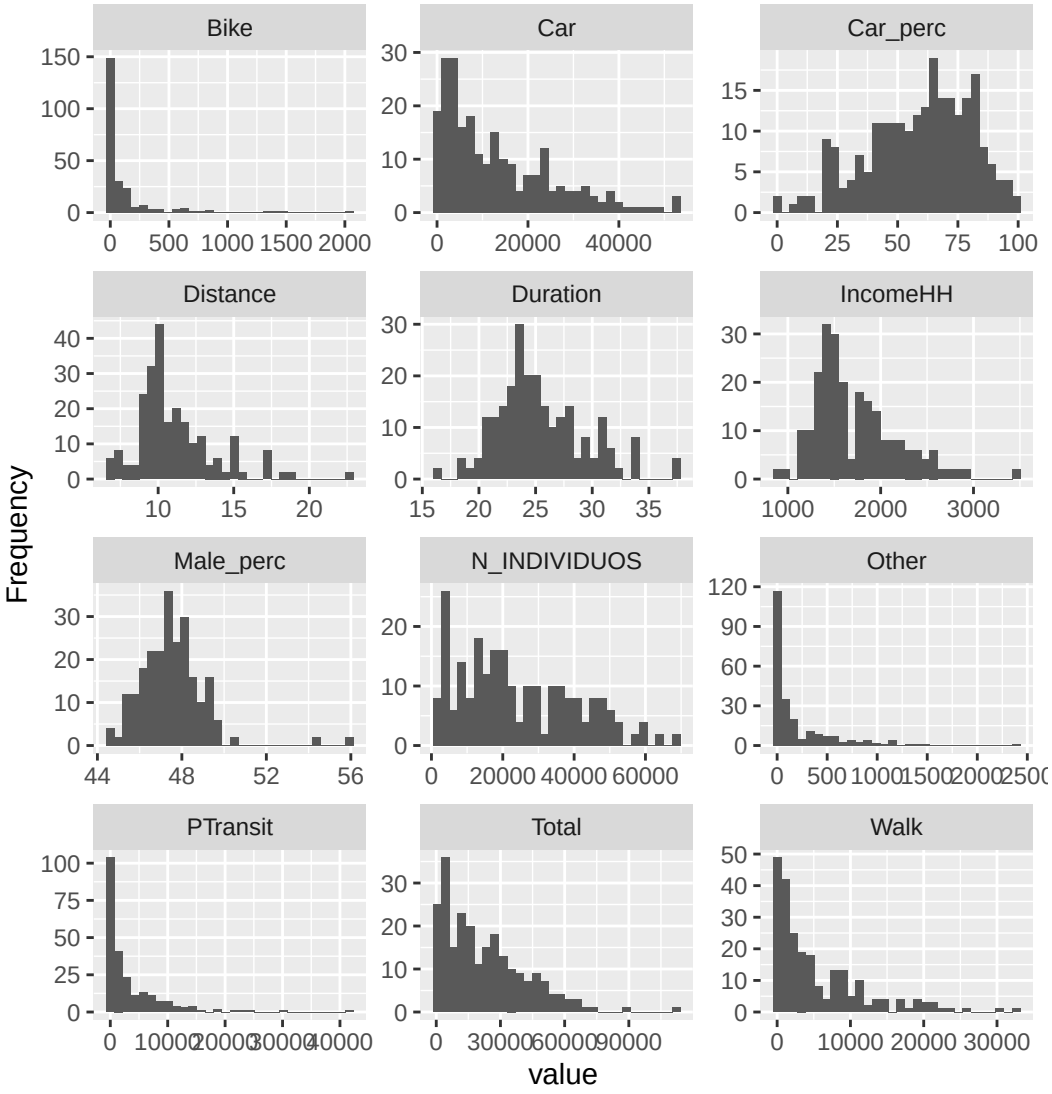
```
[1] 17739.39
```

> **Note**
>
> There are many methods to treat outliers. This is just one of them.

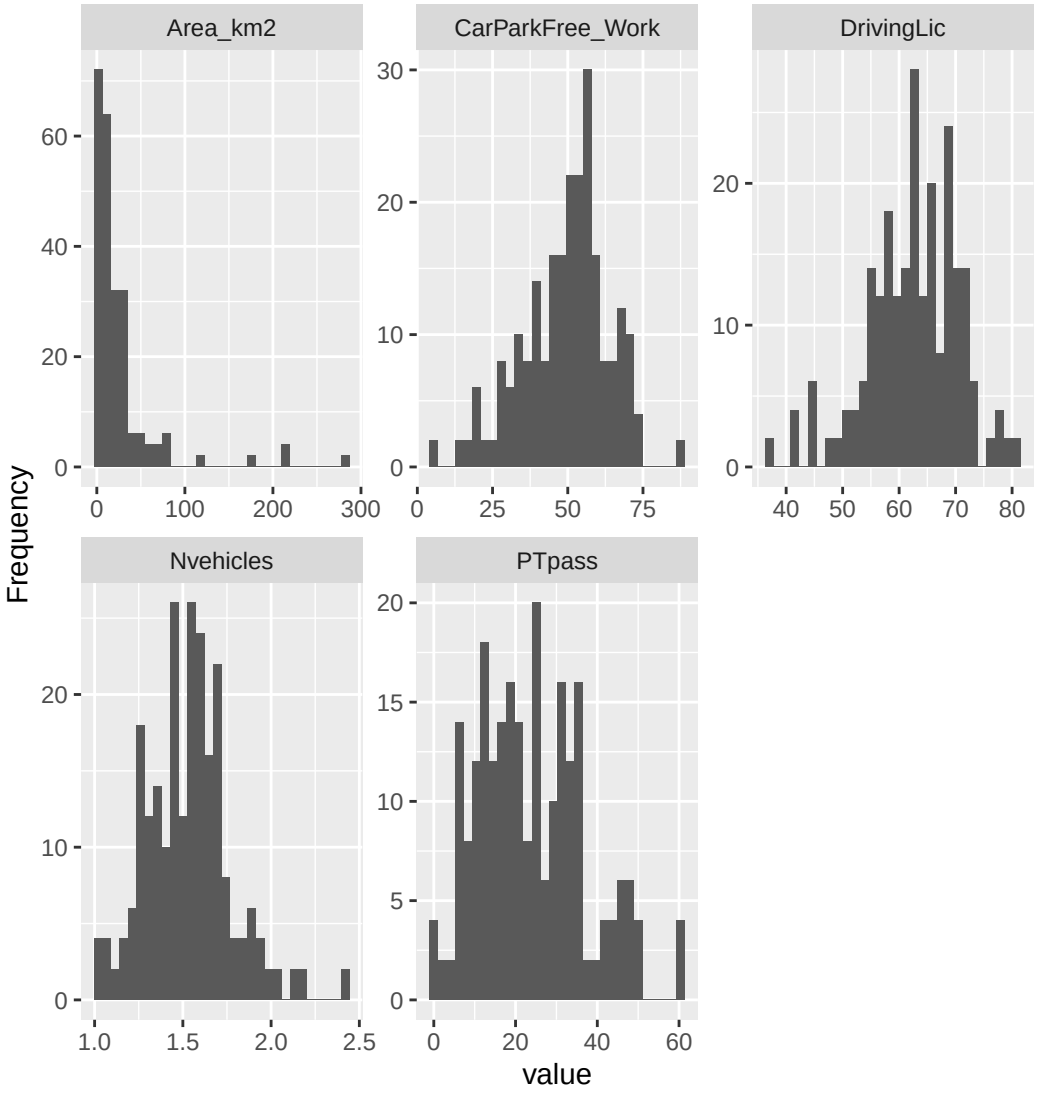
6.7. Histograms

Plot histograms of all the continuous variables

```
plot_histogram(data, ncol = 3) # histograms with 3 columns
```

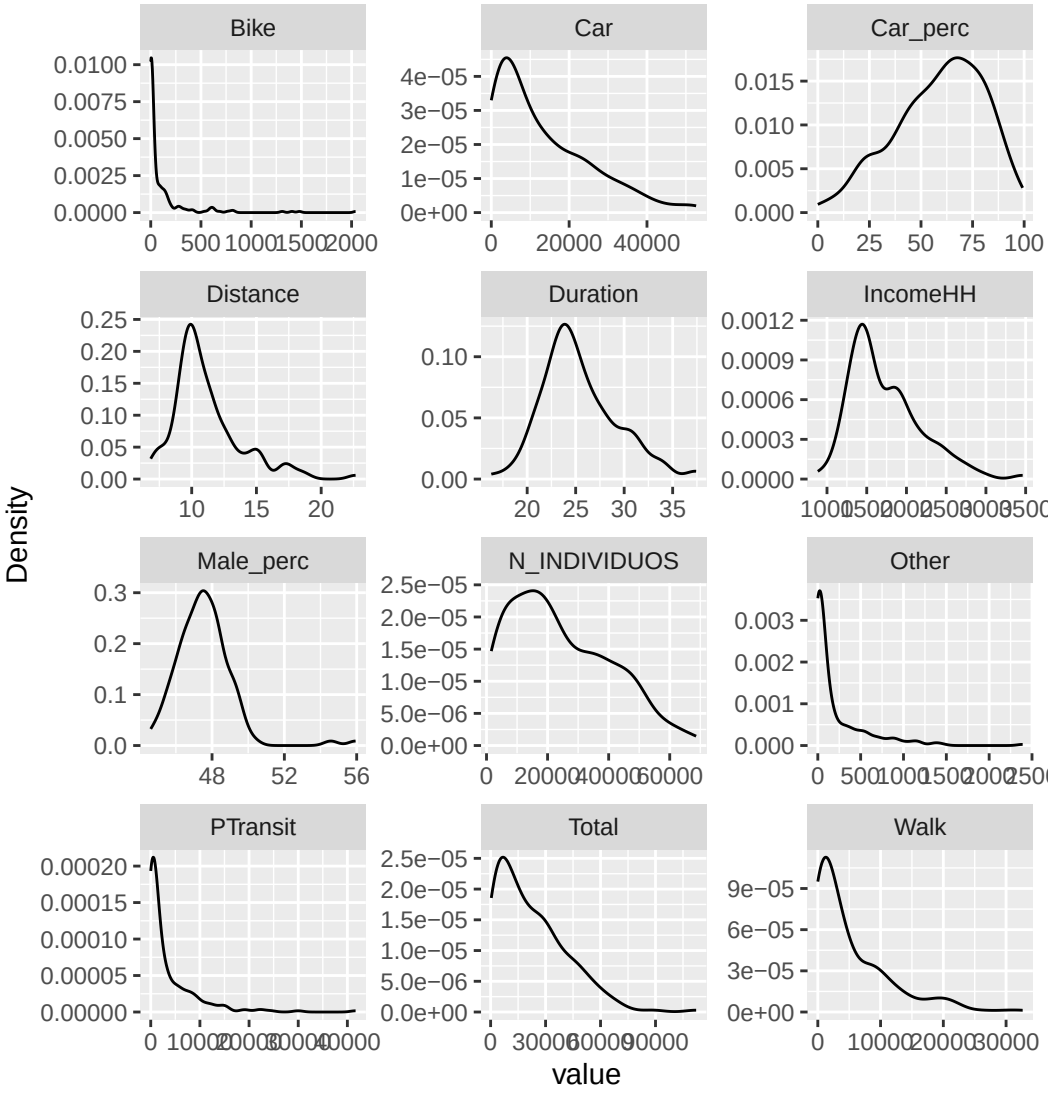

Bike
Car
Car_perc
Distance
Duration
IncomeHH
Male_perc
N_INDIVIDUOS
Other
PTransit
Total
Walk
Frequency
value

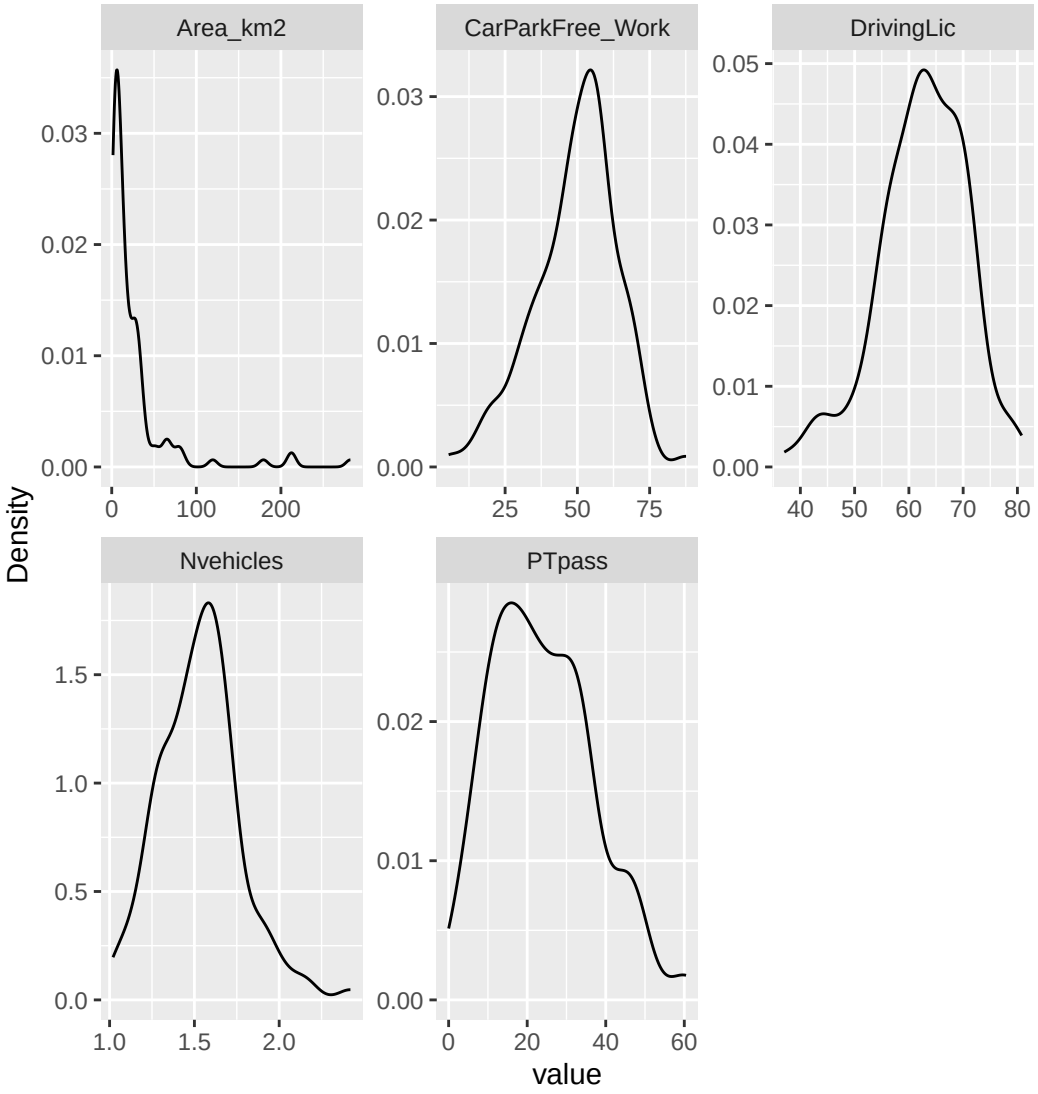
Page 1



Page 2

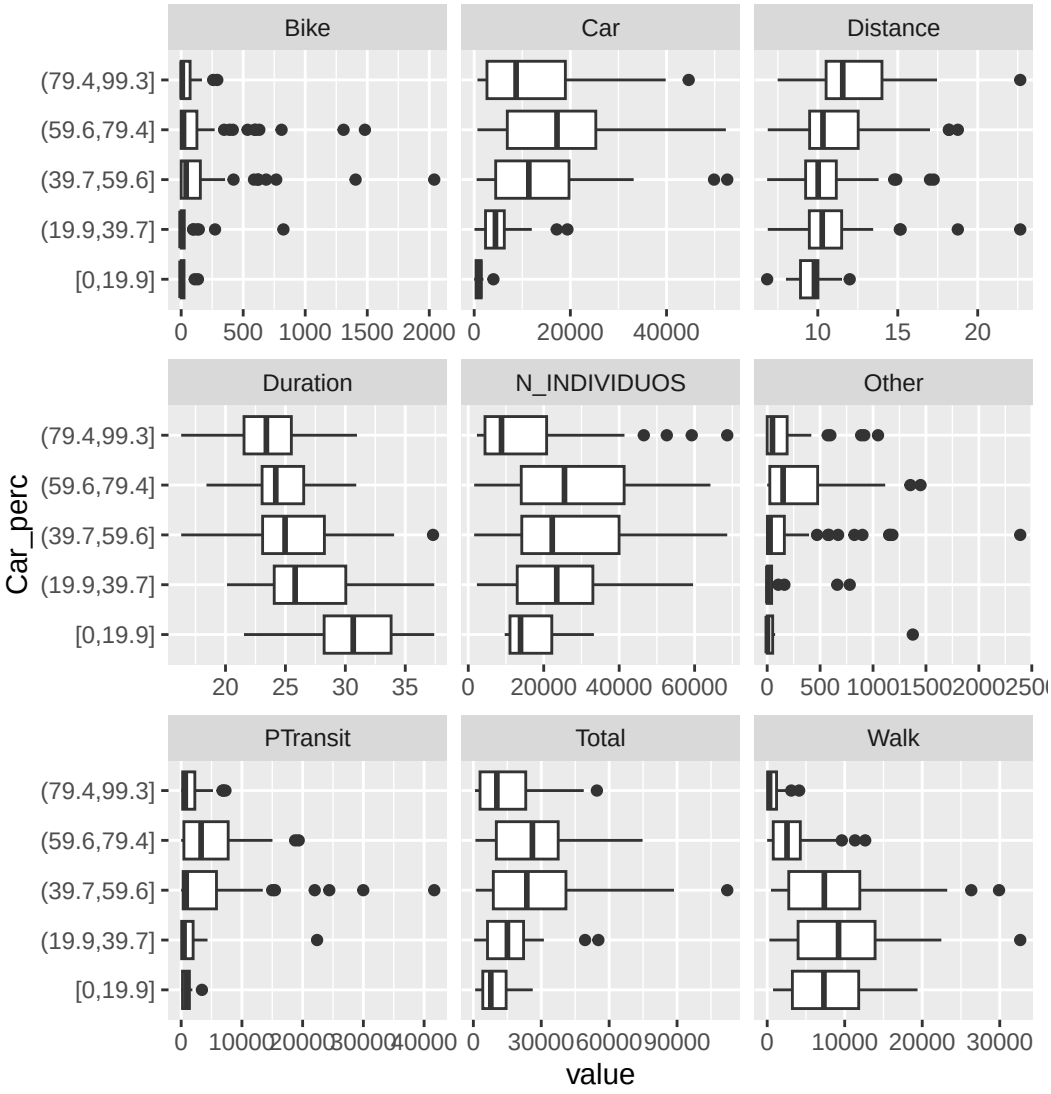
```
plot_density(data, ncol = 3) # density plots
```

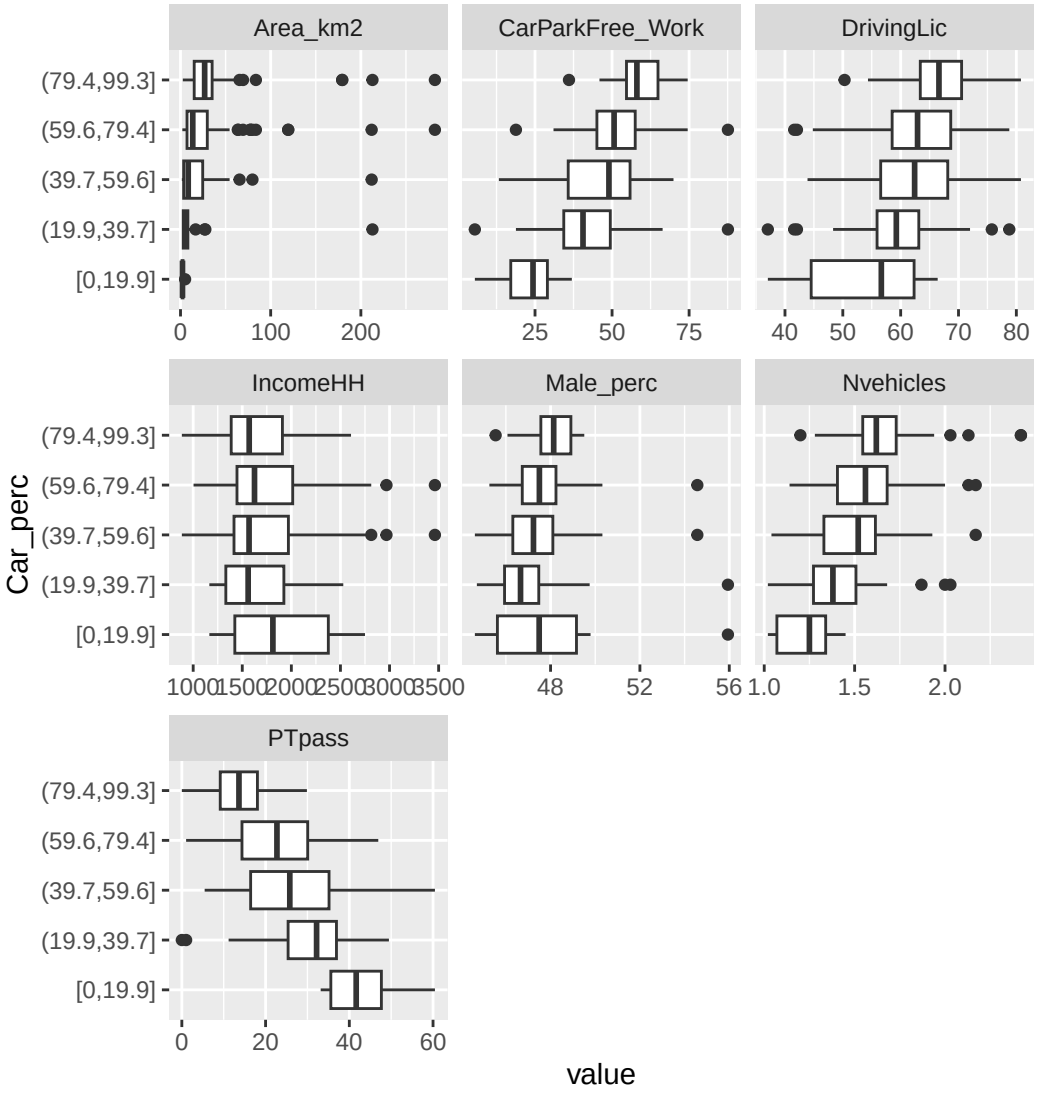

Bike
Car
Car_perc
Distance
Duration
IncomeHH
Male_perc
N_INDIVIDUOS
Other
PTransit
Total
Walk
Density
value
Page 1



Check how other variables are distributed regarding `Car_perc`

```
plot_boxplot(data, by = "Car_perc", ncol = 3)
```


Bike
Car
Distance
Duration
N_INDIVIDUOS
Other
PTransit
Total
Walk
Car_perc
(79.4,99.3]
(59.6,79.4]
(39.7,59.6]
(19.9,39.7]
[0,19.9]
value
Page 1



Note

When `Car_perc` increases, `PTpass` decreases.

Exercise

Try plotting the boxplots of each independent variable with `Car_perc`.

6.8. Correlations

Plot correlation heatmaps, between continuous variables.

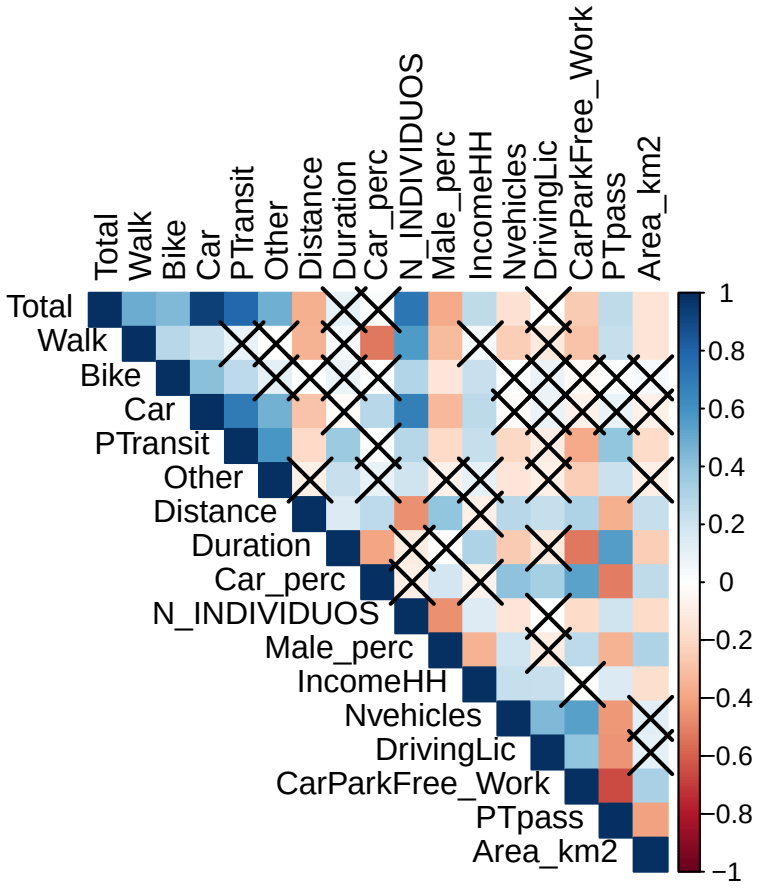
```
# estimate correlation matrix
corrmat = cor(data_continuous, method = "pearson") |> round(2)
corrmat
```

```
                Total  Walk  Bike   Car PTransit Other Distance Duration
Total            1.00  0.49  0.44  0.93     0.78  0.48    -0.36     0.10
Walk             0.49  1.00  0.27  0.21     0.09 -0.01    -0.34     0.03
Bike             0.44  0.27  1.00  0.41     0.26  0.11     0.00     0.08
Car              0.93  0.21  0.41  1.00     0.70  0.47    -0.29    -0.03
PTransit         0.78  0.09  0.26  0.70     1.00  0.58    -0.20     0.36
Other            0.48 -0.01  0.11  0.47     0.58  1.00    -0.09     0.22
Distance        -0.36 -0.34  0.00 -0.29    -0.20 -0.09     1.00     0.16
Duration         0.10  0.03  0.08 -0.03     0.36  0.22     0.16     1.00
Car_perc         0.01 -0.53  0.00  0.27     0.01  0.09     0.26    -0.40
N_INDIVIDUOS     0.72  0.56  0.29  0.68     0.28  0.20    -0.46    -0.11
Male_perc       -0.38 -0.32 -0.14 -0.33    -0.20 -0.08     0.39    -0.01
IncomeHH         0.25  0.04  0.22  0.26     0.23  0.10    -0.09     0.30
Nvehicles       -0.16 -0.25  0.00 -0.02    -0.21 -0.14     0.27    -0.26
DrivingLic      -0.01 -0.12  0.08  0.08    -0.10 -0.09     0.23    -0.12
CarParkFree_Work -0.26 -0.29 -0.03 -0.08    -0.38 -0.25     0.30    -0.53
PTpass           0.25  0.23  0.01  0.09     0.39  0.21    -0.36     0.55
Area_km2        -0.15 -0.15  0.03 -0.08    -0.19 -0.09     0.23    -0.25
                Car_perc N_INDIVIDUOS Male_perc IncomeHH Nvehicles DrivingLic
Total               0.01         0.72     -0.38     0.25     -0.16      -
0.01
Walk               -0.53         0.56     -0.32     0.04     -0.25      -
0.12
Bike                0.00         0.29     -0.14     0.22      0.00       0.08
Car                 0.27         0.68     -0.33     0.26     -0.02       0.08
PTransit            0.01         0.28     -0.20     0.23     -0.21      -
0.10
Other               0.09         0.20     -0.08     0.10     -0.14      -
0.09
Distance            0.26        -0.46      0.39    -0.09      0.27       0.23
Duration           -0.40        -0.11     -0.01     0.30     -0.26      -
0.12
Car_perc            1.00        -0.09      0.18    -0.07      0.40       0.33
N_INDIVIDUOS       -0.09         1.00     -0.46     0.14     -0.14       0.00
Male_perc           0.18        -0.46      1.00    -0.34      0.19      -
0.11
IncomeHH           -0.07         0.14     -0.34     1.00      0.23       0.22
Nvehicles           0.40        -0.14      0.19     0.23      1.00       0.44
DrivingLic          0.33         0.00     -0.11     0.22      0.44       1.00
CarParkFree_Work    0.53        -0.19      0.26    -0.01      0.54       0.39
PTpass             -0.52         0.20     -0.34     0.16     -0.44      -
0.45
```

```
Area_km2              0.25        -0.19      0.30    -0.18       0.12        0.10
                CarParkFree_Work PTpass Area_km2
Total                    -0.26   0.25    -0.15
Walk                     -0.29   0.23    -0.15
Bike                     -0.03   0.01     0.03
Car                      -0.08   0.09    -0.08
PTransit                 -0.38   0.39    -0.19
Other                    -0.25   0.21    -0.09
Distance                  0.30  -0.36     0.23
Duration                 -0.53   0.55    -0.25
Car_perc                  0.53  -0.52     0.25
N_INDIVIDUOS             -0.19   0.20    -0.19
Male_perc                 0.26  -0.34     0.30
IncomeHH                 -0.01   0.16    -0.18
Nvehicles                 0.54  -0.44     0.12
DrivingLic                0.39  -0.45     0.10
CarParkFree_Work          1.00  -0.66     0.32
PTpass                   -0.66   1.00    -0.41
Area_km2                  0.32  -0.41     1.00
```

```r
# store the results so you can call the p-value at the corrplot
res = cor.mtest(data_continuous, conf.level = .95)

corrplot(
  corrmat,
  method = "color", # or "circle"
  p.mat = res$p,
  sig.level = 0.05,
  type = "upper", # display only the upper triangular
  # order = "hclust", # order by hierarchical clustering
  tl.col = "black" # text label color
)
```

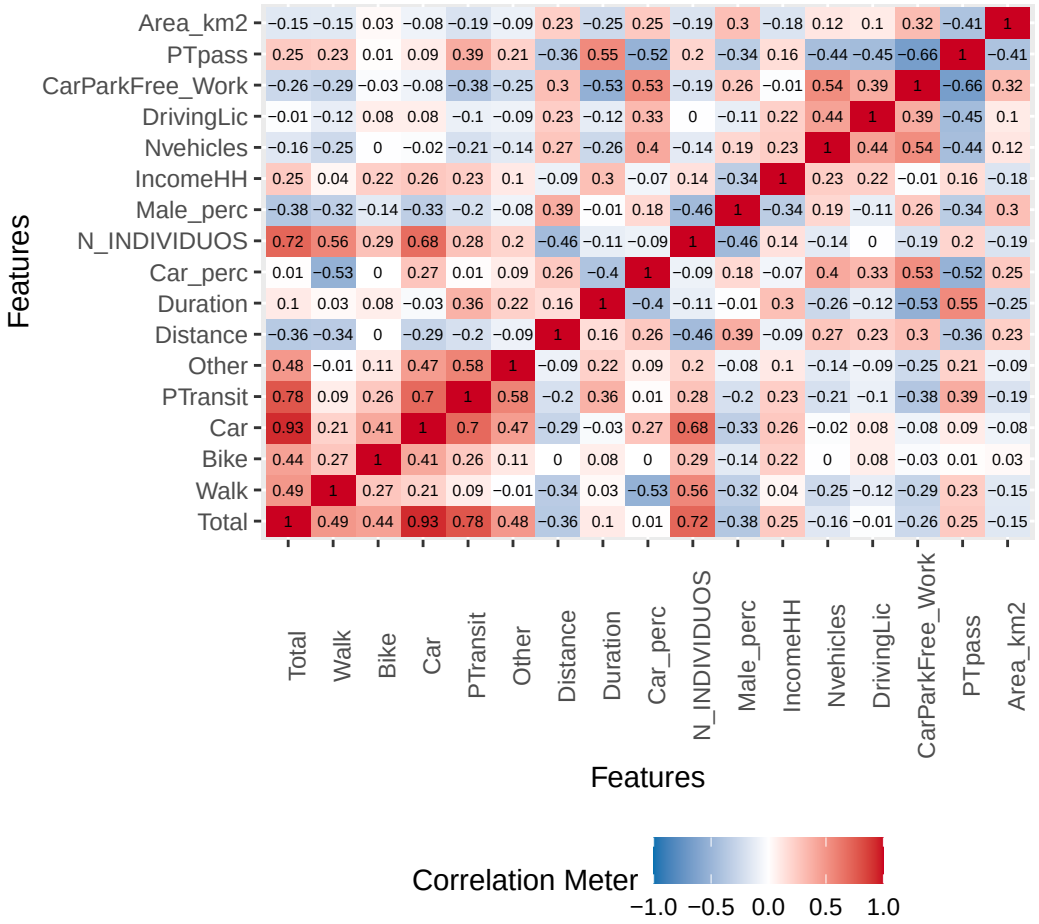


Note

The pairwise correlations that are crossed are statistically not significant. The null hypothesis (H_0) is that correlation is zero. The alternative hypothesis (H_A) is that correlation is not zero.
This means that the correlations are only significant when you reject the null hypothesis (p-value < 0.05).

Other method, using the `DataExplorer` package

```
plot_correlation(data_continuous)
```



Check the *p-value* of a crossed pair correlation:

```
cor.test(data$IncomeHH, data$Bike)
```

```
    Pearson's product-moment correlation

data:  data$IncomeHH and data$Bike
t = 3.5261, df = 234, p-value = 0.0005074
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.09977779 0.34249284
sample estimates:
      cor
0.2246163
```

```
cor.test(data$IncomeHH, data$Duration)
```

```
    Pearson's product-moment correlation

data:  data$IncomeHH and data$Duration
t = 4.817, df = 234, p-value = 2.62e-06
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.1795420 0.4122446
sample estimates:
     cor
0.300356
```

```
cor.test(data$Distance, data$Duration)
```

```
    Pearson's product-moment correlation

data:  data$Distance and data$Duration
t = 2.4605, df = 234, p-value = 0.0146
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.0317533 0.2808153
sample estimates:
      cor
0.1588098
```

The default for `cor.test` is Pearson, two-sided, with a 95% confident level. Check `?cor.test` for more options.

References

Engel, Claudia A. 2023. *Introduction to r*. cengel.github.io/R-intro/.

INE. 2018. *Mobilidade e Funcionalidade Do Território Nas Áreas Metropolitanas do Porto e de Lisboa: 2017*. Instituto National de Estatística. https://www.ine.pt/xportal/xmain?xpid=INE&xpgid=ine_publicacoes&PUBLICACOESpub_boui=349495406&PUBLICACOESmodo=2&xlang=pt.

INE. 2022. *Censos 2021- XVI Recenseamento Geral da População. VI Recenseamento Geral da Habitação*. Instituto National de Estatística. https://censos.ine.pt/xurl/pub/65586079.

Lovelace, Robin, Jakub Nowosad, and Jannes Muenchow. 2024. *Geocomputation with r*. 2nd ed. Chapman; Hall/CRC. https://r.geocompx.org/.

Wickham, Hadley, Mine Çetinaka-Rundel, and Garrett Grolemund. 2017. *R for Data Science*. 2nd ed. O'Reilly Sebastopol. https://r4ds.hadley.nz/.